

Systems integration throughout the early life cycle

W M Pryke, P J Whiting, S Pang, R Kaur and C Wilmot

This paper presents various methodologies and approaches for performing systems integration and testing throughout the early and middle stages of the development life cycle. It discusses techniques for estimation and management of the integration early life cycle. Prevention and early detection of errors significantly reduces the cost of failure in a system. The systematic use of quality gates throughout the life cycle reduces the propagation of problems. The appropriate use of documentation standards ensures information and knowledge is disseminated to all those who require it. These and other techniques for effective control of integration and test are discussed, together with techniques for measuring the effectiveness of systems integration, such as test coverage analysis, metrics, and post-delivery assessment.

1. Introduction

This paper discusses the systems integration activities that should take place during the early development life cycle. Following a brief introduction to these activities and why they should be performed, certain major aspects are discussed in more detail, namely:

- planning and estimation,
- requirements capture,
- quality gates,
- test coverage and selection,
- documentation and configuration management,
- metrics that can be used to measure success.

1.1 Why test?

A question often asked is why test at all. The answer is management of risk. No one is perfect and with the best will in the world errors are going to be introduced at every stage in the development life cycle (Fig 1).

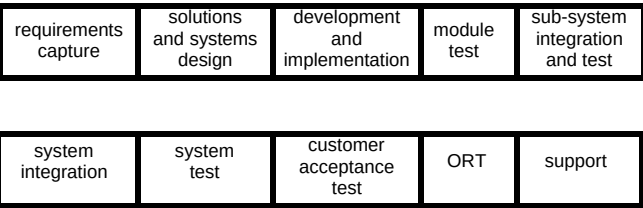


Fig 1 The development life cycle.

The consequence of not finding these errors is the cost of failure. This cost may be direct, the cost of rectification or fallback, but there may also be indirect costs such as corporate embarrassment, legal or regulatory consequences, or simply loss of future revenue from an affected customer.

The question is how big is the risk and how much testing is needed to reduce that risk to an acceptable level. The answer is by using analytical techniques to quantify the complexity and associated risk. So how can the inherent risk of errors in a system be reduced? The solution is to introduce a systematic systems integration approach throughout the life cycle. This paper describes the outline methodology employed within BT generally, and discusses tools and techniques used within Systems Integration department to provide a whole life cycle integrated risk management service.

The earlier an error is identified, the less it costs to fix that error (Fig 2). An error found in deployment typically costs a thousand times more to fix than if found at the 'requirements' stage.

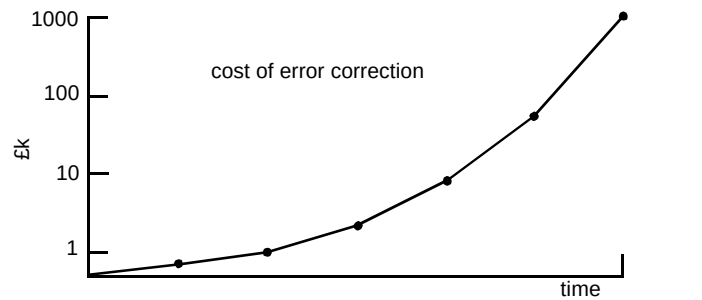


Fig 2 The cost of error correction over time.

Therefore testing should start as early as possible, preferably at the 'requirements specification' stage. A good set of requirements is a testable set of requirements. This means that the requirements must be complete, self-consistent, and fully resolved and defined. By performing a testability analysis on the requirements set prior to sign off, such problems as ambiguous, contradictory, missing or incomplete requirements will be removed, which will reduce errors further downstream. The designers, the developers, the testers and the customer will all have the same specification of the system being produced. The process of requirements capture is discussed later (see section 3).

Subsequent involvement in the life cycle may be illustrated using the 'V' model (Fig 3).

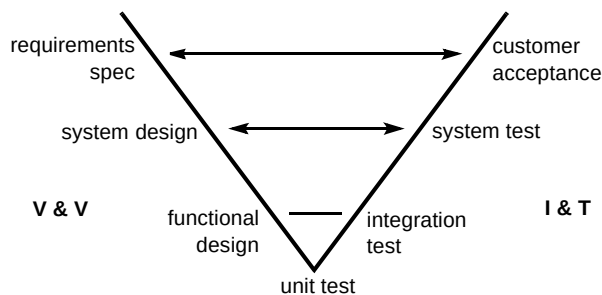


Fig 3 The 'V' Model.

Testing can be broken down into two interrelated sections — verification and validation (V&V) and integration and test (I&T), each represented by an arm of the 'V' in Fig 3.

Each stage on the left side of the V corresponds to a related I&T function on the right. Customer acceptance testing is performed against the system requirements specification. Similarly system testing and integration testing are performed against the system design and functional design specifications respectively. Planning and preparation for each I&T function should start with their earlier life cycle counterparts.

What is meant by V&V? Boehm [1] offers the following definitions:

Verification: *Are we building the product right?*

Validation: *Are we building the right product?*

In other words testability analysis of the system requirements, and review and sign-off of design and functional specifications against these requirements ensure that we are building the right product. This is the validation aspect of early life cycle testing. Ensuring that system development methodologies such as code walkthroughs,

inspections and reviews are used, and appropriate unit and module testing is performed, verifies that we are building the product 'right'.

After release from development into testing, having successfully passed through a 'quality gate' (see section 5), the task breaks down into four traditional phases.

- Sub-system integration and test (SSIT)

Sub-system build and interface testing, sometimes known as grey box testing — the internal interfaces between modules comprising the final build of a sub-system or intermediate sub-build are tested.

- System integration

This is known as black box testing as no knowledge is assumed as to the internal workings of the system. Testing takes the form of supplying a known and controlled stimulus to the system and verifying the correctness of the resulting response. This verification is made against the original system requirements.

- System test

The process of testing integrated software and hardware to verify that it meets the specified system design — this testing is based on customer scenarios and workstring processes for provisioning, faulting, etc. This testing also covers non-functional testing such as security and performance [2].

- Customer acceptance test

This is the proving of the system to the customer, quite often involving the re-running of a subset of the system tests. Other terms for this phase include user acceptance testing (UAT) or final verification office (FVO).

All the foregoing assumes the traditional waterfall model of development and test. Fast track and rapid application (RAD) type developments are becoming more common, which dictate a totally different approach to systems integration. This RAD view of systems integration is described by Carter et al [3].

2. Planning and estimation

Planning and estimation is one of the most important enablers for the success of an integration project. How much time and money is required to address the risk of failure? Planning an integration task is the act of setting the balance in the cost, time, and quality triangle (see Fig 4).

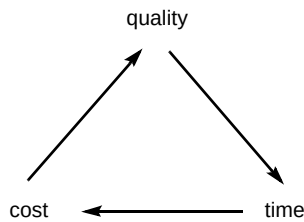


Fig 4 The cost triangle.

The expense of underestimating the complexity and therefore cost of integration can be as expensive as overestimating (Fig 5).

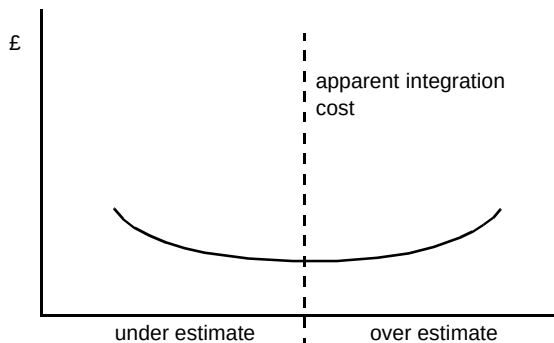


Fig 5 The results of poor estimation.

The difficulty is that estimation and planning need to be undertaken during the requirements stage of a project where sufficient detailed information is generally not known across all systems. This makes the task of assessing the complexity of the integration task a difficult one. The first objective must be to try to quantify the task at hand. It is important to define the work breakdown structure and integration life cycle for the project and then to start to assess the complexity of each phase.

When trying to set the 'time/cost/quality' balances there are really only two variables — cost and time. Quality is a variable but not one that can be quantified or varied at this stage of the life cycle.

2.1 Estimation

Within a project some areas of work will be easier to estimate than others. To aid in the estimation of the less understood areas it is necessary to first produce an estimate of the relative complexity of the work areas. Once this has been done the estimation and planning task can be achieved with a much greater level of confidence. The following estimation process has been defined and refined over several integration projects including Number Portability, Corniche, and FeatureNet.

The stages of the integration life cycle are analysed and the systems that comprise each domain are formed into a complexity matrix to derive a relative complexity value for each of the work areas in the integration life cycle. A defined integration work breakdown structure (WBS) and

customer expectation milestone dates are required as a pre-requisite to this process.

2.2 Complexity matrices

In order to assess the relative complexity of integration of sub-systems and systems a more formal and less subjective method is required. One of the techniques employed is the use of complexity matrices shown below. The matrices are constructed for estimating the complexity of an integration area based on the changes that will be made to a system and its interfaces. This technique still relies on the manual estimation of the complexity of change as a delta to the existing complexity of a known system or interface. Each system and interface is assessed and numerical values are assigned; typical numerical values given for system changes are:

- 10 new system
- 8 major enhancements to a complex system
- 4 minor enhancements to a simple system
- 3 no enhancements to complex system (i.e. large regression test)
- 1 no enhancements to simple system (i.e. small regression test)

Typical numerical values given for interworking/interface changes are:

- 10 new system to new system interface/ interworking
- 7 new system to old system interface/ interworking
- 7 old system to new system interface/ interworking
- 5 impact of interface/ interworking somewhere in between
- 2 old system to old system interface /interworking

2.3 Formulating matrices

For sub-system integration testing (intra-domain testing), e.g. Billing SSIT, the same systems appear on the x-axis and y-axis of the matrix with the delta to their relative complexity alongside them (Fig 6).

Billing SSIT

bill FDS	0					
CBP	0	0				
NMP	2					
RT-SDP	0					
SDP	3	8				
Tx server	8	0	2	0	10	8
	0	0	2	0	3	8
	bill FDS	CBP	NMP	RT-SDP	SDP	Tx server
complexity total	8	8	48	8	251	8
total	331					

Fig 6 The SSIT complexity matrix.

The delta to complexity for interface appears with the cells of the matrix where an interface exists. The formulae

used to derive the complexity value for SSIT is based on both the interworking and the individual system complexities for each point of intersection of the matrix. Thus the following formulae should be used:

$$C_{AB} = (\text{Sys A comp}) \times (\text{Sys B comp}) \times (\text{Interwork comp}) + (\text{Sys A comp}) + (\text{Sys B comp})$$

Total complexity of SSIT test plan is the sum of the calculated complexities:

$$C_{\text{TOTAL}} = C_{AB} + C_{BC} + C_{CD} \dots \text{etc.}$$

For systems integration testing (inter-domain testing), e.g. network management/service management SIT, the systems from the two domains appear separately on either the x-axis or y-axis of the matrix with the delta to their relative complexity alongside them (Fig 7).

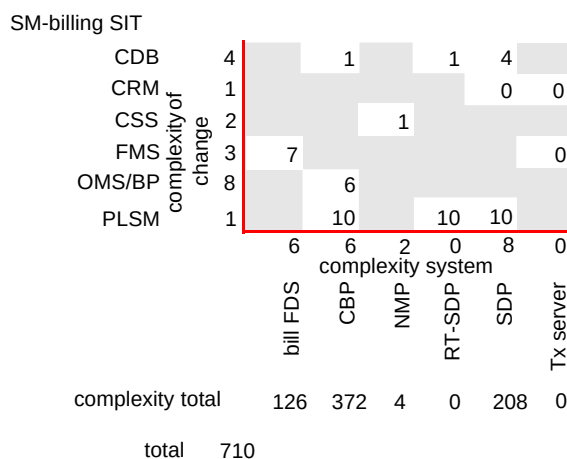


Fig 7 SIT complexity matrix.

The delta to complexity for interface appears with the cells of the matrix where an interface exists. The formulae used to derive the complexity value for SIT is based on only the interworking and complexity for each point of intersection of the matrix. Thus the following formulae should be used:

$$C_{AB} = (\text{Sys A comp}) \times (\text{Sys B comp}) \times (\text{Interwork comp})$$

The total complexity of SIT test plan is the sum of the calculated complexities:

$$C_{\text{TOTAL}} = C_{AB} + C_{BC} + C_{CD} \dots \text{etc.}$$

For system testing it is assumed that the resources from the SSIT activities are available for system test preparation and that all the test resource is available to work on system test execution.

2.4 Planning

It is important to put together a Gantt view of the integration tasks with input milestones for delivery dates for all components into SSIT and customer-expected-delivery milestones. It is then possible to derive the start of integration, i.e. when two or more systems are available, start of system test and expected delivery date.

Taking the duration of the tasks and the complexity values a manual estimation should be attempted to identify the resource numbers for each of the integration phases. This exercise should be undertaken by a number of experienced integration managers and their results aggregated.

Once resource levels are agreed these values are included in the plan and first-draft resource demand forecasts can be derived. Resource levelling will need to be applied and the final values will need to be tempered with the constraints of resource availability.

2.5 Balance

All estimation information should be held in a plan and baselined for future analysis. This plan will give the base 'time, cost, quality' balance that has to be agreed by the customer. As time goes on the pressures on the triangle heave and yaw; only by accurately tracking the plan can the true cost of change be measured either as a change in time, cost, or quality.

3. The early life cycle

Verification and validation constitutes the early life cycle phase of a waterfall life cycle. In essence V&V is considered as the 'testing' of the requirements and design specifications at the start of the life cycle, where the emphasis is on adding value to requirements and design, not only through conventional methods such as QA reviews, walk-throughs and inspections, but also through dynamic matrix mapping methods.

It has been stated above that early life cycle activity concentrates on the prevention of errors and anomalies at the requirements and design stages, and this is achieved successfully in practice through requirements, testability, and traceability analyses.

3.1 Requirements analysis

This analysis is an essential part of V&V. The main activity is conducted to check the validity of the requirement specifications to see that the requirements are complete, consistent, unambiguous and meet the systems objectives. The requirements capture and analysis process is explained in more detail later, including evaluation against

hardware, user, operator and user-interface requirements documentation.

3.2 Testability analysis

As the name suggests this activity concentrates on test aspects of each individual requirement. The reviewer asks if the statements of the requirements can be tested or demonstrated. Can the requirement be written in more quantifiable terms? It can be argued that if there are no means of demonstrating a requirement then in effect it is not being fulfilled. The importance of this early testability will aid the production of user scenarios and test-case definitions into the integration and test stages of the life cycle.

3.3 Traceability analysis

This activity concentrates on tracking the requirements from capture to design specification. Traceability is aided by uniquely identifying each of the requirements and harnessing them in a database, so that each one will map to design specifications, test cases, etc. As is often the case, there will exist a one-to-one and one-to-many relationship in the matrix mapping methods.

Use of matrix methods has proved successful in large, complex intelligent network projects where a change in requirements could be mapped and tracked through the life cycle.

Early life cycle activity has been found to be very productive. The early life cycle team is able to produce V&V reports or testability reports to show the results of the above analyses and identify critical paths and risks for correction at these early stages. Other advantages that may be gained from early life cycle activity are:

- early visibility of design documentation so that test planning may commence at the earliest opportunity, i.e. user scenario production and preparation for test model, etc.
- bringing together design, development and test communities to focus on customer requirements throughout, whereas before the prime objective may have become somewhat hazed and confused as each party solely concentrated on its own activity — this also helps to break down walls in the life cycle processes.

Once issues are identified, it is important for the test team and design community to work closely to reach an agreed resolution. If no suitable resolution can be found then a problem report will be raised.

4. Requirements capture and analysis

There are two types of requirement — functional and non-functional. Functional requirements define what the system shall do. Non-functional requirements define the attributes of a system as it does its job, together with resource limitations and constraints. Functional requirements can be regarded as the ‘what’, and the non-functional as the ‘how’. A good requirement should not contain its own solution. It should be correct, unambiguous, atomic, verifiable, traceable, consistent with other requirements and easy to understand.

Requirements work has always been seen as an overhead. However, unless requirements are adequately known and specified, it is very difficult to assess the suitability of the solution, and it is virtually impossible for the systems integration and test team to perform any form of structured testing for the product delivered.

One of the most important roles in this process is the requirements manager or consultant. This person is charged with maintaining the ‘requirements’ baseline and ensuring all project members understand them. Projects that fail to employ suitable requirement management will at best deliver a badly conceived solution that will most likely fail to meet the customer’s needs.

4.1 The process

A simplified view of the requirements capture process is described in Fig 8, an adaptation of Fig 5 in the Requirements Management Handbook [4].

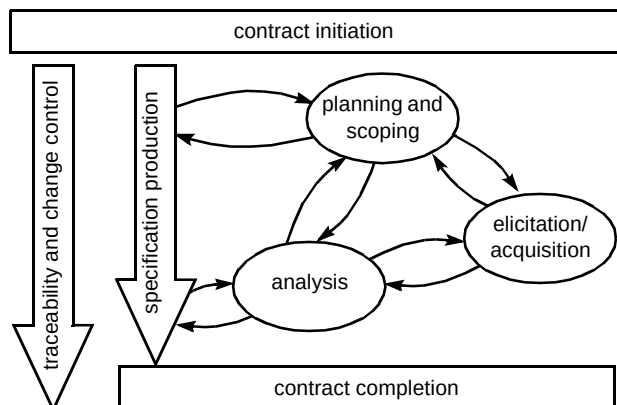


Fig 8 Requirements capture process (simplified).

Requirements capture and analysis is an iterative process. It is usually initiated by a business change, or driven by competitive threat or regulation. Sometimes the trigger is a technological breakthrough or new ideas.

Requirements capture and analysis activities start with initiating a contract between the customer and the

'requirements' team. This defines the scope and costing of the task. Other activities include negotiation roles and responsibilities, identifying a customer authority and agreeing deliverables, milestones and cost. To maintain the flexibility and to cope with the rate of change, the task definition and scope need to be reviewed frequently so that the contract can continue to reflect the need of the customer.

In the planning and scoping stage, an initial understanding of the problem is identified and a real business need should be confirmed. At this stage, project objectives, risks, critical success factors and deliverables are identified. An understanding of who should be involved in order to gather information is also established. Resourcing, and project and quality planning should also take place at this stage.

After the scope of the project has been established, elicitation/acquisition activities will then take place. These include identifying who needs to be consulted and why, identifying types of information to be collected and gathering the information. Information gathering approaches may involve document analysis, interviews, workshops, questionnaires, and brown paper (process walk-through) techniques [4].

In the analysis stage, data and the gathered information are collated and converted into requirements. Everyone in the project should agree the meaning of each requirement. Quite often, requirements from different sources will be conflicting; negotiation to reach a consensus is important. It is essential that a requirement consultant is appointed to ensure that those affected by the project (customers, users, developers and testers) maintain a realistic view of what will be delivered or achieved as the result of the requirement exercise. Prioritising requirements can be used to achieve that.

The integration team should be involved in this stage to verify that the requirements identified are unambiguous, complete, consistent and testable. Ideally the integration team should work alongside the requirement team or work as part of the team. As a minimum, they should be involved in reviewing the requirements. The system integration view is then taken into account and acceptance test planning can be initiated.

Other activities in the analysis may also include modelling requirements or prototyping to form an abstract representation of data or processes that serve as an aid to the understanding of the requirements. During the analysis, it is not unusual that the scope of the project may change. In this case, the cycle of planning and scoping reiterates. A requirements specification document should be produced at the end of the analysis. This set of requirements should be agreed by the customers before further design work is carried out.

Once the requirements document has been agreed, this then forms the basis on which the design team can base their design work. The integration team can start their test plans and verify the design against these requirements. Any changes to the requirements are then dealt with via a change control process. Changes will be evaluated by the design and system integration teams to establish the impact caused. Relevant changes will then be made to the design and testing, if appropriate. Following a change control process will help the project to maintain the traceability of requirements and the configuration management of the product delivered. Traceability and change control on the requirements should be performed throughout the development life cycle.

4.2 Requirements management

After the requirements have been captured and analysed, it is essential that they be managed properly throughout the development life cycle. Effective requirements management [4] can help to ensure that:

- the whole project team understands what the requirements mean,
- ambiguities in the high-level requirements document are reduced,
- the impact of a request to change a high-level requirement can be assessed more easily across the entire project,
- requirements can be audited quickly and easily (to demonstrate how they are met, in what part of the design and within which test case it will be validated),
- deriving a statement of compliance between the finished product and the original requirements specified becomes straightforward.

The best practice is to have a single repository of all current requirements throughout the life of the project. As requirements change, traceability can be maintained. Project managers are able to control the expectations of the customers. The systems integration team can effectively estimate the effort required to perform systems integration activities and are able to assess the impact on the testing required and time-scales for any changes of requirement. There are requirements management tools available to aid this process. However, these tend to be complicated and not user-friendly. Using a simple database, like Microsoft[®] Access, has proven sufficient for a number of projects.

5. Quality gates

A quality gate is, as its name suggests, a gateway that clearly delineates phases in a development life cycle and provides the opportunity to audit and maintain appropriate quality levels (i.e. fitness for purpose) at those points. As a result the probability of errors being propagated further are greatly reduced, thus reducing the cost of failure.

Quality gates frequently receive a bad press and are often cited as being wasteful in terms of time. However, experience on several development-type projects has shown the success of involving relevant people from the requirements capture, development, and deployment teams. Involvement should take place as early as possible in the planning and formulation of quality gates so that they are viewed as a united team goal and not just an impediment from the test team.

There are six quality gates recommended as a minimum for integration:

- end of requirements definition,
- end of solution design,
- end of system design,
- end of component/module test,
- end of integration test,
- end of system test.

6. Test coverage and test case selection

Myers [5] states that a necessary part of every test case is a definition of the expected result. This may seem an obvious statement but the experience shows that the expected result is often only vaguely specified or omitted altogether. A detailed description of the expected outcome for a given stimulus:

- reduces human perception errors,
- makes the tester think more carefully about the test,
- makes regression testing easier and more efficient,
- is a must for test automation [6].

The coverage of a set of test cases is dependent on the overall testing requirement, given that time and financial constraints now dictate that 'gold plate' or complete and thorough testing is inappropriate in most cases. For example, a newly developed system will need to be tested in a different way to an off-the-shelf product or to a re-release of an existing system. For a totally new system it is appropriate to spend time testing for incorrect functionality as well as expected functionality, and to test for invalid or inappropriate stimuli. For existing products or systems, Petschenik [7] offers the following advice:

- testing a system's capabilities is more important than testing its components (i.e. test the functionality the customer will actually use),
- testing old capabilities is more important than testing new ones (i.e. the customer should be no worse off than before re-release),
- testing typical situations is more important than testing boundary cases.

Berard [8] suggests a mechanism for deciding the effectiveness of test case selection. He makes the two following definitions:

- a Good Test Case is one that has a high probability of detecting an as yet undiscovered error,
- a Successful Test Case is one that detects an as yet undiscovered error.

Within systems integration the product 'Requirement Matrix' is used as a measure of potential effectiveness to classify test cases as being potentially 'Good', and thus given a higher priority if time and budget constraints are in force. Each test case designed to directly test a particular requirement is deemed to be a 'Good' test case. As a measure of the efficacy of this classification an analysis of corresponding test results is used to identify the 'Successful' cases.

When considering test coverage the following questions arise.

- How do we know that there are test cases to cover everything that should be covered?
- Are some areas being tested too much and others too little?
- How do we prove to ourselves and to our customers that we have got it right?

In systems integration the Requirements Matrix is again used to demonstrate that every requirement has a corresponding test case(s), but in the end the experience and engineering judgement of the SI team is invaluable.

Weyuker [9] presents a much more mathematically rigorous, axiomatic approach for checking completeness of testing effort.

7. Project documentation and configuration management

The need for a complete and consistent documentation set is real but not always obvious. The textbook reasons are as follows:

- to demonstrate the quality of the project to auditors and to show customers that the job is being done correctly,
- to offer traceability at all points back to original requirements and to ensure test coverage,
- to pass information from one stage to another.

In practice, however, the third point is the most critical. It is essential that all parties (not just the integration team) have enough information to be able to do their job effectively. Experience shows that more time and effort is wasted by not having any form of requirements list available to the integration team than is gained by the 'just do it' approach, where documentation is given low priority.

As mentioned above, effective test-case design and proof rely on the availability of (at least) a coherent set of requirements.

Templates for the preferred documentation set may be found in BT's VV&T Handbook [10].

Configuration management includes document control, build control, and problem reporting.

The successful management of each is vital for a successful project. Within Systems Integration a centralised 'document control' function has been most successful, enabling project engineers to concentrate on their primary function.

With regard to the management of problem reporting, best practice is seen to be the use of a Change Control Board (CCB) for medium and large complexity projects. The CCB should consist of a representative from each relevant project area (for example, design, development, test, support), a customer representative and be chaired by an impartial manager, with sufficient authority to adjudicate. This system of problem and change management has worked effectively on a wide range of projects.

Build and version control from the systems integration point-of-view is most effective when performed locally. Not only does this ensure that the test team know at all times the version of the product or system they are testing, but the build process enables an element of initial training on the product or system, giving a warm start to testing proper.

8. Metrics

Metrics are objective numerical measures extracted from the products and processes of a software or systems project. Guidance on the collecting and analysis of metrics can be found in BT's SID Metrics Handbook [11]. It is important not to take metrics as an absolute measure of quantity or quality. However, when reviewed against other similar metrics they can act as an important indicator showing trends which allow the identification of potential hot-spot areas.

Testability reports that are reviewed at the quality gates formally relay the V&V metrics to the customer, this level of information enables them to monitor and control the project.

Metrics gathered as part of the V&V analysis are, typically, testability issues and traceability issues, both of which are fed back to the product owner via QA reviews or during walk-throughs. Planning metrics should be taken from an accurately tracked plan; important things to look for are input milestone trends and high resource

concentration when undertaking V&V analysis of specific areas of design or requirements. Milestone trends can be plotted on a 'milestone trend' chart (Fig 9), which quickly identifies where dates are unstable and where there are areas of concern.

Within the Cambridge programme, early involvement in the analysis of requirements and designs and proactive resolution has enabled the integration teams to move confidently into test preparation.

For one Cambridge release 93 issues were raised against the requirements and 453 issues were raised with the design, this was not a reflection of poor design but of systematic analysis of the very complex subject. By the time the overall testability report was submitted for the solution design quality gate, there were no major outstanding problems.

When planning metrics are analysed in conjunction with the V&V metrics, their combined result allows the identification of areas where the design was least well defined (and therefore most issues were raised) and required most effort to get resolved. Early indicators show that the areas of the design which raised the most V&V issues and took most time to resolve will be those which will be the most complex to integrate. Full analysis of this subject will be more fully analysed in a post-delivery assessment.

8.1 Post-delivery assessment

Some post-delivery analysis of the relative costs have been undertaken on small integration projects to measure the effectiveness of system integration, including the early life cycle work. To date no systematic analysis of the integration costs has been undertaken for major programmes, e.g. Cashless Service Platform or Cambridge. This has largely been due to the lack of a common planning and tracking process up until one or two years ago.

Major integration programmes within BT's Networks and Systems division have only come about in the last 4 years and each typically takes 18 months to 2 years to mature. Common planning and tracking processes are now allowing the resources against detailed planned activities to be recorded and for the plans across the different specialist integration areas to be rolled up into overall plans at higher levels of abstraction. To achieve this, the PMS II planning standard is being followed.

When this is achieved, a true comparative set of values is available that enables estimates to be refined and integration programmes to be categorised (Fig 10).

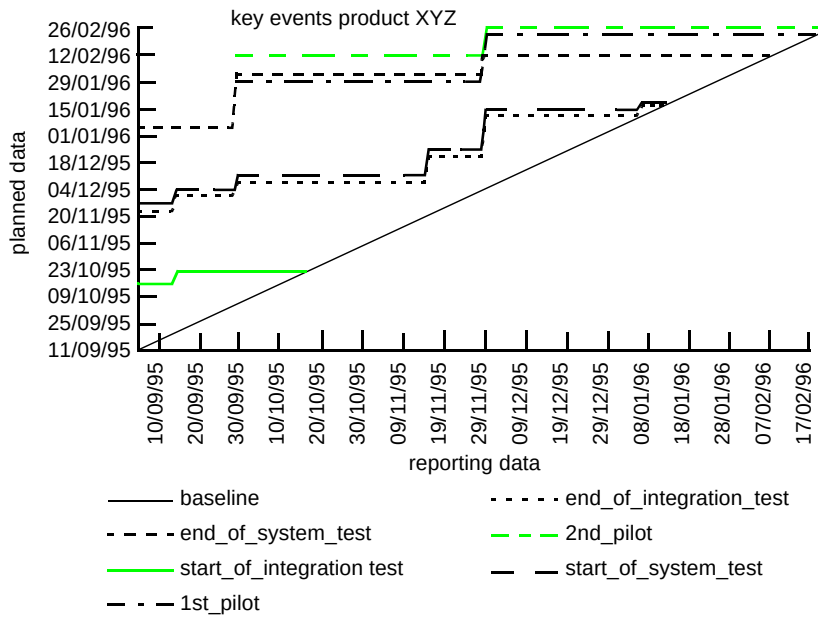


Fig 9 Milestone trend chart.

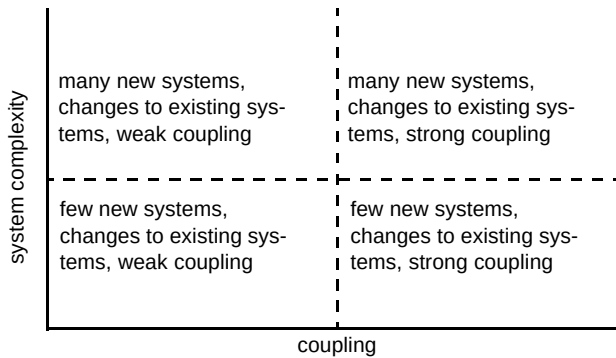


Fig 10 Classification of integration programmes.

Measurements have been taken on small projects, for example, Teleworking, and these have revealed that the integration and test costs throughout the life cycle were as shown in Table 1.

Table 1 Life cycle test costs.

Early life cycle (strategy, V&V, reference model design, and familiarisation)	26.5%
Test preparation (test designs and test cases)	47.3%
Test execution (running the tests)	26.2%

These values have been adapted, and used in the previously described estimation method, to construct estimates for larger integration programmes. The project revealed that for a simple project with few bug-fixes and limited in-life enhancements, 75% of the integration costs were in the preparation.

A high-level overview of a number of integration projects has shown that integration costs as a percentage of overall programme costs vary. As a rule of thumb, integration could typically account for 30 to 35% of the overall programme costs (see Fig 11).

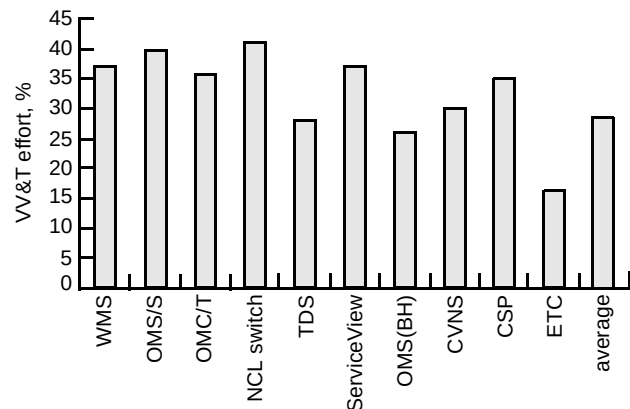


Fig 11 Integration costs by programme.

9. Conclusions

The cost of integration must always be assessed against the risk and subsequent cost of failure. The cost of failure must include both the direct loss of revenue of the service and loss of consumer confidence.

The early life cycle work is the first stage of the whole life cycle risk management service offered by Systems Integration. The importance of the whole life cycle approach is that SI provides the glue that cements and enables the delivery of integrated solutions. The quality gate approach ensures continuous monitoring of progress to

defined criteria. Through early life cycle initiatives, Systems Integration has proved that it is much more than the hurdle at the end of the project as was once perceived.

Customer satisfaction is achieved when the integration activity delivers on time, to budget, to the agreed quality criteria. To maximise the benefits of an integration-led approach the test teams must be involved in the project from the outset and the benefits understood by all areas within the project.

Feedback from recent early life-cycle activities undertaken on large-scale integration projects has demonstrated the value of this approach. This aim is not just to satisfy the customer but to delight them! By combining best-practice analytical techniques and sound project management skills, Systems Integration offers a complete managed service, forming an essential ingredient to the future of BT's proven delivery record.

References

- 1 Boehm B: 'Software engineering economics', Prentice Hall (1981).
- 2 Whiting P and Wilmot C: 'Validation testing — improving product quality', British Telecommunications, Eng J, 8, No 4 (1990).
- 3 Carter C J et al: 'Rapid integration and testing of business solutions', BT Technol J, 15, No 3, pp 37—47 (July 1997).
- 4 BT: 'Requirements Management Handbook, Issue 1', (1995).
- 5 Myers G: 'The art of software testing', Wiley (1979).
- 6 Hicks I D et al: 'Automated testing as an aid to systems integration', BT Technol J, 15, No 3, pp 26—36 (July 1997).
- 7 Petschenik N: 'Practical priorities in system testing', IEEE Software 2, No 5 (1985).
- 8 Berard E: 'Essays on object oriented software engineering', Prentice Hall (1993).
- 9 Weyuker E: 'Axiomatising software test data accuracy', IEEE Transactions on Software Engineering, SE-12, No 12 (1986).
- 10 BT Systems Integration Department (SID): 'VV&T Handbook, Issue 3', (1996).
- 11 BT Systems Integration Department (SID): 'SID Metrics Handbook', (1996).



Paul Whiting read Mathematics at the University of Birmingham, graduating in 1980. He remained at Birmingham and went on to receive an MSc in Computer Science (1981), and continued working for a PhD until 1984. Since joining BT the same year, he has worked primarily in the area of system test and integration on a wide range of projects from integrated circuits to the latest multimedia applications. He currently leads a group specialising in multimedia and on-line application integration. He is a Chartered Engineer and a member of the British Computer Society.

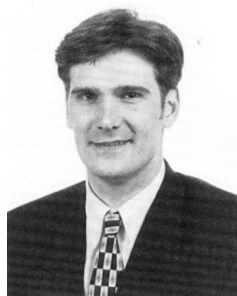


Rubi Kaur joined BT in Systems Integration in 1995 where she has been part of an early life cycle and SIT team for intelligent networks systems. She is currently performing systems integration on the BT intranet project. She graduated from Sunderland University with a BSc in Applied Physics and Computer Science and gained an MSc in Optoelectronics and Optical Information Processing at Queens' University of Belfast in 1994. Prior to joining BT she has worked at the Laser Facility of the Rutherford Appleton Laboratories on a variety of international collaborative projects and co-authored several research papers in the field of X-ray laser generation for UV plasma sources. She is a Member of the British Computer Society.



Christopher Wilmot read joint honours Electronic Engineering and Physics at the University of Birmingham. On graduating in 1981, he went to work for Plessey Telecommunications Limited. He studied for a MEng degree in Microelectronic Systems at the University of Liverpool, which he received in 1982.

He joined BT Laboratories in 1984. He has worked on numerous integration programmes including Concert Virtual Network Services and Cambridge system and integration test.



William Pryke joined BT as a trainee technician in 1983, gaining an HNC in Electronic Engineering at Suffolk College in 1987. Following his apprenticeship he joined the open systems testing unit working on OSI test methodologies as part of the OSTC European Test Forum. He represented BT within European collaborative initiatives and in 1994 authored the AFT11i test specification as part of EWOS PT15. Later that year he joined the CVNS integration team and contributed to the successful delivery of the Global Virtual Network Services platform. Since 1995 he has worked on Corniche and Cambridge early life cycle and is currently leading the solution test team for CPR0 rules-based routing.



Sauming Pang joined BT in 1993 after graduating with a degree in Electrical and Electronic Engineering. She started her career as a requirement consultant specialising in network management operational support systems for the access network and has experience in capturing, analysing and managing requirements. She then joined the multimedia systems integration team to work on systems such as BT Interactive TV and Video on Demand. She is an associate member of the IEE.